HermesQA

HermesQA là bot tự động review pull request (PR) trên GitHub. Khi có PR mới, nó đọc phần code thay đổi, tìm lỗi bằng hai nguồn, rồi comment vào đúng dòng code có vấn đề. Nguồn thứ nhất là năm công cụ static analysis chạy trong một sandbox cô lập. Nguồn thứ hai là LLM đóng ba vai: kỹ sư phần mềm (SE), kiểm thử (QA) và vận hành (DevOps).

> HIỆN ĐANG Ở ĐÂU
>
> | | |
> |---|---|
> | ĐÃ XONG | Phần lõi chạy được từ đầu đến cuối ở chế độ offline qua dòng lệnh: đọc diff, quét static trong sandbox, LLM ba vai, lọc kết quả, sinh báo cáo. |
> | ĐÃ XONG | Thực nghiệm trên 35 PR Python và 18 PR đa ngôn ngữ (JavaScript, TypeScript, React, Go, Java). |
> | DỞ DANG | Hai lượt thực nghiệm mới nhất chưa commit và chưa đưa vào báo cáo. Bước gộp kết quả static với LLM đang làm tăng số báo nhầm. |
> | CHƯA LÀM | Chạy thử với GitHub App thật: toàn bộ nhánh webhook, hàng đợi, đăng comment lên PR chưa từng chạy đầu-cuối. |

1. Dự án nhắm đến điều gì 2. Luồng chạy từ A đến Z 3. Từng phần và mục đích 4. Thực nghiệm và kết quả 5. Giai đoạn hiện tại 6. Thuật ngữ

---

1. Dự án nhắm đến điều gì

Review code thủ công tốn thời gian của người có kinh nghiệm, chất lượng không đều giữa các reviewer, và người review dễ bỏ sót lỗi bảo mật khi đang tập trung vào logic nghiệp vụ. Hai cách tự động hoá hiện có đều thiếu một nửa:

- **Static analysis** (semgrep, ruff…) rẻ và cho kết quả giống nhau mỗi lần chạy, nhưng chỉ bắt được các mẫu lỗi đã định nghĩa sẵn. Nó không biết thay đổi này thiếu test cho trường hợp nào.
- **LLM** hiểu ngữ cảnh và nhận xét được về thiết kế, test, nhưng có thể bịa ra lỗi không tồn tại, báo sai số dòng, và tốn token.

HermesQA kết hợp cả hai, với ba điểm đồ án muốn đóng góp:

1. **Review theo ba vai.** Mỗi vai có một bộ hướng dẫn riêng (checklist, thang mức độ nghiêm trọng, quy tắc chống nói thừa), và chỉ được gọi khi PR có loại file liên quan.
2. **Sandbox an toàn.** Code trong PR đến từ người lạ nên không bao giờ được thực thi. Chỉ các công cụ static đọc nó, trong một container không có mạng.
3. **Chống LLM bịa lỗi.** Bốn lớp kiểm tra xếp chồng, trong đó có bước self-verify: LLM tự kiểm chứng lại phát hiện của chính nó.

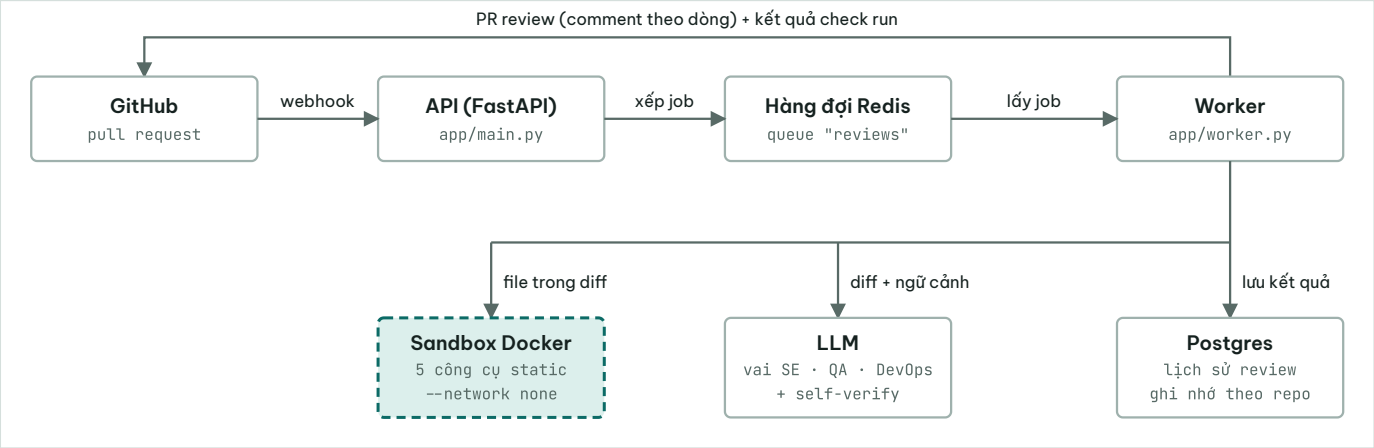
Giả thuyết cần kiểm chứng bằng thực nghiệm: **kết hợp static với LLM cho kết quả tốt hơn dùng riêng từng nguồn**. Số liệu hiện tại chưa ủng hộ giả thuyết này (xem mục 4).

Về lâu dài, README định hướng sản phẩm thành một micro-SaaS: hỗ trợ thêm GitLab và Bitbucket, dashboard web, học từ phản hồi của lập trình viên, nhiều khách hàng dùng chung và tính phí. Chưa phần

nào trong số này được bắt đầu.

2. Luồng chạy từ A đến Z

Hệ thống có ba tiến trình: một API nhận webhook, một hàng đợi Redis, và một worker làm toàn bộ việc review. Worker gọi ra ba nơi: sandbox Docker, LLM, và cơ sở dữ liệu Postgres.



Đường đi của một PR. Khung nét đứt là ranh giới cô lập: code của PR chỉ được đọc bên trong đó, không có đường ra mạng. Chế độ offline (`app/cli.py`) bỏ qua hàng trên cùng và Postgres: nó lấy diff giữa hai nhánh git ở máy local rồi chạy cùng các bước sandbox, LLM và lọc kết quả.

Mười một bước, theo đúng thứ tự trong mã

1

GitHub gửi webhook khi PR thay đổi

Khi một PR được mở, có commit mới, được mở lại, hoặc chuyển từ bản nháp sang sẵn sàng review, GitHub gọi `POST /webhook/github` kèm thông tin PR.

> **Mục đích:** bot tự chạy mỗi lần PR đổi, không ai phải bấm gì.

2

API xác thực chữ ký rồi xếp việc vào hàng đợi

`app/main.py`

API tính lại mã HMAC SHA-256 trên nội dung nhận được bằng khoá bí mật chung với GitHub. Sai chữ ký thì trả 401. Sự kiện không phải pull request và PR còn ở dạng nháp bị bỏ qua. Phần còn lại được gói thành một job (repo, số PR, mã commit, mã cài đặt App) đẩy vào hàng đợi Redis, rồi API trả lời ngay.

> **Mục đích:** GitHub cắt webhook sau 10 giây, còn một lần review mất 1–3 phút, nên phần nhận và phần xử lý buộc phải tách nhau. Chữ ký HMAC chặn người ngoài giả làm GitHub để bắt bot chạy.

3

Worker nhận job và báo "đang chạy" lên PR

`app/worker.py` · `app/github_app.py`

Worker lấy job khỏi hàng đợi (giới hạn 15 phút mỗi job). Nó ký một JWT bằng private key của GitHub App để đổi lấy installation token ngắn hạn, rồi tạo một check run tên "HermesQA Review" ở trạng thái `in_progress`.

> **Mục đích:** người mở PR thấy ngay bot đang làm việc. Token ngắn hạn chỉ có quyền trên repo đã cài App, nên nếu lộ thì thiệt hại có giới hạn.

4

Clone đúng commit của PR, đọc cấu hình của repo

`app/worker.py`

Worker tải riêng commit đầu của PR với `--depth 1`, checkout, rồi gỡ remote khỏi thư mục `.git`. Sau đó đọc file `.hermesqa.yml` ở gốc repo nếu có: vai nào được bật, file nào bỏ qua, convention của team, số comment tối đa, có chặn merge khi gặp lỗi critical hay không.

> **Mục đích:** công cụ static và phần lấy ngữ cảnh cần file thật trên đĩa. Gỡ remote để token không nằm lại trong thư mục mà sandbox sắp đọc.

5

Lấy diff và lọc file

`app/review/diff_utils.py`

Worker lấy diff của PR qua GitHub API và tách thành từng file. File khớp danh sách bỏ qua, file nhị phân và file bị xoá bị loại. Với mỗi file còn lại, hệ thống ghi nhớ những số dòng phía bản mới có mặt trong diff. Nếu không còn file nào, check run kết thúc `success` và dừng ở đây.

> **Mục đích:** mọi bước sau chỉ xét phần thay đổi. Danh sách số dòng này về sau quyết định comment được phép đặt ở đâu, vì GitHub chỉ nhận comment vào dòng nằm trong diff.

6

Quét static trong sandbox

`app/sandbox.py` · `runner/run_static.sh` · `app/review/static_tools.py`

Worker ghi danh sách file thay đổi ra `targets.txt` rồi khởi chạy container `hermesqa-runner`. Bên trong, năm công cụ chạy lần lượt (semgrep, bandit, gitleaks, hadolint, ruff) và ghi kết quả JSON ra thư mục `/out`. Worker đọc các file JSON đó, chuẩn hoá về một cấu trúc chung gọi là `Finding` (file, dòng, mức độ, loại, mô tả, độ tin cậy), và chỉ giữ finding thuộc file thay đổi. Trạng thái từng công cụ (chạy được, thất bại, không áp dụng) được ghi lại để in ở đầu báo cáo.

> **Mục đích:** bắt các mẫu lỗi đã biết mà không tốn token, và cùng đầu vào thì luôn cho cùng kết quả. Sandbox bảo đảm một PR chứa mã độc không làm gì được hệ thống.

7

Chọn vai cần chạy

`app/review/router.py`

PR có sửa code ứng dụng thì gọi SE và QA. PR có đụng tới file hạ tầng (Dockerfile, workflow CI, `requirements`, `package.json`, Terraform…) thì gọi DevOps. PR chỉ sửa tài liệu thì chỉ SE xem qua. Kết quả được giao với danh sách vai bật trong `.hermesqa.yml`.

> **Mục đích:** tiết kiệm token. PR chỉ sửa Dockerfile không cần hỏi vai QA.

8

Từng vai review bằng LLM

`app/review/agent.py` · `app/skills/<vai>/SKILL.md`

Mỗi vai là một lần gọi LLM. File `SKILL.md` của vai làm system prompt. Phần câu hỏi gồm: tiêu đề và mô tả PR, convention của repo, ghi nhớ từ các lần review trước, tóm tắt những gì static đã bắt kèm yêu cầu không lặp lại, diff của từng file (tối đa 25 file) cùng 30 dòng xung quanh mỗi chỗ sửa có in sẵn số dòng, và một khuôn JSON bắt buộc cho câu trả lời. LLM trả về một câu tóm tắt và danh sách finding.

> **Mục đích:** bổ sung những thứ công cụ static không thấy được: lỗi logic, thiếu test, thiết kế kém. In sẵn số dòng và ép khuôn JSON để LLM khó báo sai vị trí và kết quả xử lý được bằng máy.

9 **Self-verify: LLM kiểm chứng lại chính nó**

`app/review/agent.py`

Nếu một vai có finding, danh sách đó được gửi lại cho cùng model kèm code, lần này với lời dặn đóng vai reviewer cấp cao: bỏ finding không phải vấn đề thật, sửa số dòng lệch, bỏ finding trùng, cập nhật độ tin cậy. Nếu lần gọi này lỗi, hệ thống giữ kết quả ban đầu.

> **Mục đích:** giảm lỗi bịa. Đây là cơ chế được coi là đóng góp chính của đồ án, nhưng thực nghiệm chưa đo được lợi ích của nó (mục 4).

10 **Gộp, lọc và ra kết luận**

`app/review/postprocess.py`

Finding của static và của LLM được gộp lại. Hai finding cùng file, lệch nhau không quá 2 dòng và cùng loại thì chỉ giữ một, ưu tiên bản của static. Tiếp theo là chính sách lọc: bỏ finding có độ tin cậy dưới 0,6 hoặc mức độ dưới `medium`; dòng được báo phải nằm trong diff (lệch tối đa 3 dòng thì kéo về dòng gần nhất); tối đa 15 comment gắn vào dòng, phần còn lại đưa vào danh sách phụ. Cuối cùng là kết luận của check run: `failure` nếu có lỗi critical và repo bật chặn merge, `neutral` nếu có lỗi critical hoặc high, còn lại `success`.

> **Mục đích:** PR không bị ngập comment, không có comment trỏ vào dòng không tồn tại, và team tự quyết bot có được chặn merge hay không.

11 **Đăng kết quả, lưu lại, dọn dẹp**

`app/review/report.py` · `app/github_app.py` · `app/memory.py`

Worker sinh báo cáo Markdown đầy đủ, đăng một PR review gồm phần tóm tắt và các comment theo dòng (yêu cầu sửa nếu kết luận là `failure`, còn lại chỉ là nhận xét), đóng check run với kết luận ở bước 10, và lưu lượt review vào Postgres. Nếu GitHub từ chối comment theo dòng, worker gửi lại chỉ phần tóm tắt. Thư mục làm việc luôn được xoá khi kết thúc. Nếu có lỗi giữa chừng, check run đóng ở trạng thái `neutral` kèm thông báo lỗi, để bot hỏng không chặn nhầm việc merge.

> **Mục đích:** kết quả xuất hiện ngay tại nơi lập trình viên đang làm việc, và có lịch sử để tra lại.

---

3. Từng phần và mục đích

Ba vai review

VAI	CÂU HỎI NÓ TRẢ LỜI	XEM NHỮNG GÌ
SE	Thay đổi này có đưa bug hoặc nợ kỹ thuật vào nhánh chính không?	Lỗi logic và điều kiện biên, bảo mật ngoài phần static đã báo, hiệu năng, xử lý lỗi, thiết kế, thay đổi làm vỡ API.
QA	Điều gì có thể làm thay đổi này hỏng khi chạy thật, và có test nào bắt được không?	Test cho phần mới, trường hợp biên còn thiếu, chất lượng test (assert yếu, test bị skip hoặc xoá), rủi ro hồi quy.
DevOps	Thay đổi này có build, deploy và chạy an toàn không?	Dockerfile, CI/CD, cấu hình và secret, Kubernetes/Terraform, dependency, khả năng vận hành.

Mỗi file `SKILL.md` dài khoảng 55 dòng, gồm checklist bắt buộc, định nghĩa bốn mức độ (critical, high, medium, low) và quy tắc chống nhiễu: tối đa 8 finding mỗi vai, không chắc thì bỏ, không góp ý phong cách cá nhân.

Năm công cụ static

CÔNG CỤ	BẮT ĐƯỢC GÌ
`semgrep`	Mẫu lỗi bảo mật và logic trên nhiều ngôn ngữ (SQL injection, command injection…). Chỉ nạp bộ luật của ngôn ngữ có mặt trong diff.
`bandit`	Lỗi bảo mật riêng của Python (`shell=True`, `eval`, MD5…).
`gitleaks`	Khoá API và mật khẩu bị viết thẳng vào code.
`hadolint`	Lỗi và thói quen xấu trong Dockerfile.
`ruff`	Lỗi Python từ hình thức (import thừa) đến lỗi thật (biến chưa định nghĩa, so sánh `None` bằng `==`, `except` trống). Nhóm sau được xếp loại bug thay vì style.

Sandbox: vì sao code của PR không làm hại được hệ thống

RỦI RO	BIỆN PHÁP TRONG MÃ
Code độc trong PR được chạy	Không bước nào thực thi code của PR. Công cụ static chỉ đọc file.
Công cụ bị lợi dụng để gửi dữ liệu ra ngoài	Container chạy với `--network none`. Hệ quả: bộ luật semgrep phải nạp sẵn vào image lúc build.
Leo thang đặc quyền	Bỏ toàn bộ capability, bật `no-new-privileges`.
Vắt kiệt tài nguyên máy chủ	Giới hạn 2 CPU, 2 GB RAM, 256 tiến trình, tối đa 600 giây.
Sửa mã nguồn trong lúc phân tích	Thư mục mã nguồn được gắn ở chế độ chỉ đọc.
Lộ token GitHub	Token ngắn hạn, gỡ khỏi `.git` ngay sau khi clone.
Webhook giả	Chữ ký HMAC SHA-256, so sánh bằng `compare_digest` để tránh dò theo thời gian phản hồi.

Bốn lớp chống LLM bịa lỗi

1. **Ràng buộc đầu vào.** Prompt in sẵn số dòng thật bên cạnh từng dòng code và yêu cầu chỉ báo dòng nằm trong diff.
2. **Self-verify.** Lượt gọi thứ hai để loại finding sai và sửa số dòng (bước 9).
3. **Kiểm tra số dòng bằng mã.** Finding trỏ vào dòng ngoài diff quá 3 dòng không được gắn comment, chỉ nằm trong danh sách phụ.
4. **Ưu tiên static khi trùng.** Hai nguồn cùng báo một chỗ thì giữ bản của static.

Ngoài ra còn ngưỡng độ tin cậy 0,6 và trần 15 comment mỗi PR. Lớp 1, 3 và trần comment là kiểm tra tất định. Lớp 2 và lớp 4 là hai chỗ thực nghiệm cho thấy chưa hiệu quả.

Bản đồ mã nguồn

Khoảng 1.000 dòng Python trong `app/`, 1.100 dòng cho bộ thực nghiệm trong `eval/`, và 220 dòng test.

FILE	VIỆC CỦA NÓ
`app/main.py`	Nhận webhook, kiểm chữ ký, xếp job.
`app/worker.py`	Điều phối toàn bộ một lần review (bước 3 đến 11).
`app/cli.py`	Chạy cùng pipeline trên repo local, không cần GitHub. Dùng để phát triển và chạy thực nghiệm.

FILE	VIỆC CỦA NÓ
`app/github_app.py`	Xác thực GitHub App, lấy diff, đăng review, tạo và đóng check run.
`app/sandbox.py`	Khởi chạy container quét static với các ràng buộc an toàn.
`app/memory.py`	Lưu lịch sử review và ghi nhớ theo repo vào Postgres.
`app/review/agent.py`	Dựng prompt, gọi LLM, ép câu trả lời về JSON, self-verify. Có hai backend: API kiểu OpenAI và Hermes Agent qua dòng lệnh.
`app/review/diff_utils.py`	Tách diff, tính dòng nào được comment, lấy ngữ cảnh quanh chỗ sửa.
`app/review/router.py`	Chọn vai theo loại file.
`app/review/static_tools.py`	Chuẩn hoá kết quả năm công cụ, báo công cụ nào thất bại.
`app/review/postprocess.py`	Gộp trùng, áp chính sách lọc, soạn comment, ra kết luận.
`app/review/report.py`	Sinh báo cáo Markdown và phần tóm tắt đăng lên PR.
`runner/`	Image sandbox và script chạy năm công cụ.
`eval/`	Sinh dataset, chạy thực nghiệm, chấm điểm.

---

4. Thực nghiệm và kết quả

Không có bộ dữ liệu chuẩn cho bài toán này nên đồ án tự sinh. `eval/make_dataset.py` tạo một repo Python sạch và 35 nhánh, mỗi nhánh cài đúng một lỗi đã biết (tổng 36 vị trí lỗi: 13 bảo mật, 8 DevOps, 7 bug, 4 test, 4 style và hiệu năng). Vị trí lỗi được tính tự động thành đáp án. `eval/run_eval.py` chạy HermesQA trên từng nhánh theo từng cấu hình, và `eval/score.py` so kết quả với đáp án.

Một finding được tính là đúng khi cùng file, lệch không quá 3 dòng và đúng loại lỗi so với một đáp án chưa được khớp. Mọi finding còn lại bị tính là báo nhầm, kể cả khi nó là nhận xét hợp lý nằm ngoài đáp án. Vì vậy precision ở đây là cận dưới. Khi chạy thực nghiệm, ngưỡng lọc của sản phẩm được nới hết để đo khả năng phát hiện.

Bộ Python: 35 PR, 36 lỗi

CẤU HÌNH	ĐÚNG	BÁO NHẦM	BỎ SÓT	PRECISION	RECALL	F1	GIÂY/PR	TOKEN/PR
`static` · chỉ công cụ static	23	26	13	46,9 %	63,9 %	54,1 %	101,4	0
`llm` · chỉ LLM ba vai	34	27	2	55,7 %	94,4 %	70,1 %	13,3	7.229
`both` · kết hợp (mặc định của sản phẩm)	35	61	1	36,5 %	97,2 %	53,0 %	119,2	7.552
`llm_noverify` · LLM, tắt self-verify	34	28	2	54,8 %	94,4 %	69,4 %	7,0	3.324
`llm` chạy lại lần hai	33	26	3	55,9 %	91,7 %	69,5 %	19,9	7.163

Số liệu được tính lại ngày 01/10/2026 từ các file kết quả đang có trên máy, bằng chính hàm chấm của `eval/score.py`. Hai lượt có ghi chú: ở `both`, case `truthy-or-bug` bị ghi nhận quá thời gian nên giây/PR tính trên 34 lượt; ở lần chạy lại `llm`, case `cmd-shell-true` có một vai bị lỗi nên kết quả case đó không đầy đủ.

Bắt được theo nhóm lỗi (bộ Python)

NHÓM	STATIC	LLM	BOTH
Bảo mật	12/13	12/13	12/13

NHÓM	STATIC	LLM	BOTH
DevOps	6/8	8/8	8/8
Bug	4/7	6/7	7/7
Test	0/4	4/4	4/4
Style	1/3	3/3	3/3
Hiệu năng	0/1	1/1	1/1

Bộ đa ngôn ngữ: 18 PR (JavaScript 5, TypeScript 4, Go 4, Java 3, React 2)

CẤU HÌNH	ĐÚNG	BÁO NHẦM	BỎ SÓT	PRECISION	RECALL	F1	GIÂY/PR	TOKEN/PR
`static`	6	1	12	85,7 %	33,3 %	48,0 %	89,9	0
`llm`	18	13	0	58,1 %	100 %	73,5 %	33,6	8.336
`both`	18	16	0	52,9 %	100 %	69,2 %	96,2	8.570

Ngoài Python và Dockerfile, static chỉ bắt được lỗi bảo mật (6/7) và không bắt được lỗi nào thuộc nhóm bug, test hay hiệu năng (0/11). Chỉ semgrep và gitleaks áp dụng được cho các ngôn ngữ này.

Kết quả sau đợt cải tiến 05–07/10/2026 (học từ Open Code Review)

Sau khi khảo sát Open Code Review của Alibaba, đồ án triển khai bảy cải tiến: ghim nhận xét bằng đoạn code thay vì số dòng, thay self-verify bằng bước kiểm tra sự thật có phần mềm đối chiếu, danh sách kiểm tra theo loại file kèm mục "không báo", chia lô thay vì cắt bớt im lặng, loại trừ file bí mật/sinh tự động, dấu vân tay nhận xét giữa các lần đẩy, và giám khảo AI phân loại báo động giả. Chạy lại toàn bộ với cùng mô hình (`gemini-3.1-flash-lite`). Các bảng phía trên là số **trước** cải tiến (01/10 và 05/10); bảng dưới là số **sau**. "Đúng dòng" là chấm ở độ lệch 0 dòng.

CẤU HÌNH	ĐÚNG	BÁO NHẦM	BỎ SÓT	PRECISION	RECALL	F1 TRƯỚC	F1 SAU	F1 ĐÚNG DÒNG
Python · `static` (rescore lần chạy gốc)	21	9	15	70,0 %	58,3 %	54,1 %	63,6 %	54,5 %
Python · `llm` (kiểm tra sự thật)	34	6	2	85,0 %	94,4 %	70,1 %	89,5 %	89,5 %
Python · `llm_noverify`	34	6	2	85,0 %	94,4 %	69,4 %	89,5 %	89,5 %
Python · `llm_legacyverify` (self-verify cũ)	33	11	3	75,0 %	91,7 %	70,1 %	82,5 %	82,5 %
Python · `both` (mặc định sản phẩm)	34	8	2	81,0 %	94,4 %	66,0 %	87,2 %	76,9 %
Đa ngôn ngữ · `llm`	18	6	0	75,0 %	100 %	73,5 %	85,7 %	76,2 %
Đa ngôn ngữ · `both`	18	8	0	69,2 %	100 %	72,0 %	81,8 %	72,7 %

Số `static` và `both` Python là sau khi gộp mọi cảnh báo tĩnh cùng dòng thành một comment (`static` không chạy lại vì tất định: rescore kết quả gốc bằng hậu xử lý hiện tại; mất 2 lỗi đúng vì nhãn của cụm lấy theo cảnh báo chính) (trước bước gộp này: 35 đúng, 15 báo nhầm, F1 81,4 %). Thời gian và token mỗi PR: `llm` 18,5 s · 6.355 token (không kiểm chứng 12,2 s · 4.532; self-verify cũ 18,6 s · 8.656), `both` 118 s · 6.551; đa ngôn ngữ `llm` 18,3 s · 6.994, `both` 110 s · 7.052. Giám khảo AI (`gemini-3.5-flash-lite` , 3 phiếu) phân loại báo nhầm của `llm` : Python 6/6 là nhận xét đúng nằm ngoài đáp án, đa ngôn ngữ 5/6 đúng và 1 bịa.

Thử trên pull request thật (AACR-Bench, 15 PR Python, 68 vấn đề do kỹ sư xác nhận)

Đây là phép thử cho nghi ngờ "điểm cao vì danh sách kiểm tra được viết sau khi nhìn thấy bộ lỗi tiêm". Chấm theo cách của bài báo AACR-Bench: giám khảo AI bỏ 3 phiếu xác nhận nhận xét của hệ thống và nhận xét của kỹ sư nói về cùng một vấn đề. Mô hình `gemini-3.5-flash-lite`.

CẤU HÌNH	NHẬN XÉT	TRÚNG	PRECISION	RECALL	F1
`llm`	8	1	12,5 %	1,5 %	2,6 %
`both` như cũ	286	2	0,7 %	2,9 %	1,1 %
`both`, chỉ giữ cảnh báo tĩnh trong phạm vi thay đổi	43	1	2,3 %	1,5 %	1,8 %

> **Trên code thật, hệ thống gần như im lặng**
>
> 15 PR chỉ sinh 8 nhận xét (không vai nào lỗi, bước kiểm tra sự thật không loại gì), recall 1,5–3 % so với ~90 % trên lỗi tiêm. Hệ thống tốt nhất trong bài báo AACR-Bench (agent dùng Claude 4.5 Sonnet) đạt recall 10,1 %. Con số 89,5 % vì thế phải đọc là "trên bộ đề tự tạo", không phải năng lực thực tế; đây là hạn chế được nói thẳng trong báo cáo.

> **Công cụ tĩnh trên repo thật chủ yếu báo về code không đổi**
>
> 240/275 cảnh báo tĩnh nằm ngoài phần sửa của PR; GitHub cũng không cho đặt comment lên những dòng đó. Bộ lọc "chỉ trong phạm vi thay đổi" được thêm vào sản phẩm: số cảnh báo 286 → 43, mất 1 lần trúng vốn không đăng được.

Số liệu nói gì

> **Kết hợp static với LLM chưa tốt hơn dùng riêng LLM**
>
> Trên bộ Python, `both` bắt thêm được đúng một lỗi so với `llm` (35 so với 34) nhưng số báo nhầm tăng từ 27 lên 61, kéo F1 từ 70,1 % xuống 53,0 %, và chậm hơn khoảng 9 lần. Bộ đa ngôn ngữ cho cùng chiều: F1 73,5 % xuống 69,2 %. Đây là kết quả ngược với giả thuyết ban đầu của đồ án.

> **Bước gộp trùng gần như không gộp được gì**
>
> Số báo nhầm của `both` (61) còn cao hơn tổng của hai nguồn khi chạy riêng (26 + 27 = 53). Điều kiện gộp hiện tại đòi hai finding phải cùng loại lỗi theo đúng từng chữ, trong khi static và LLM có thể gán loại khác nhau cho cùng một chỗ. Đây là việc kỹ thuật cần sửa trước tiên.

> **Chưa đo được lợi ích của self-verify**
>
> Tắt self-verify, F1 là 69,4 %. Bật, F1 là 70,1 %. Chạy lại cấu hình có bật thêm một lần cho 69,5 %. Khoảng chênh giữa bật và tắt bằng với dao động giữa hai lần chạy giống hệt nhau, trong khi bật self-verify tốn gấp đôi token (7.229 so với 3.324) và gần gấp đôi thời gian. Mỗi cấu hình mới chạy một hoặc hai lần nên chưa thể kết luận self-verify vô ích, nhưng cũng chưa có bằng chứng nó có ích. Lưu ý rằng bước này dùng chính model và chính vai đã sinh ra finding.

> **LLM bắt được nhiều hơn static ở hầu hết các nhóm lỗi**
>
> Hai nguồn ngang nhau ở nhóm bảo mật (12/13). Ở các nhóm còn lại LLM đều hơn, rõ nhất là test (4/4 so với 0/4). Static vẫn có đóng góp riêng: sau khi ruff được sửa, nó bắt được lỗi so sánh `None` bằng `==` mà lượt `llm` gốc bỏ sót.

> **Static chậm hơn LLM nhiều lần**
>
> Khoảng 100 giây mỗi PR so với 13 giây. Trước khi tối ưu (chỉ quét file trong diff, chỉ nạp luật của ngôn ngữ liên quan) con số là khoảng 350 giây.

> **Một lỗi "bỏ sót" là do cách chấm**
>
> Case `secret-aws` có hai đáp án ở hai dòng liền nhau (access key và secret key). Công cụ báo một finding bao trùm cả cặp, nhưng mỗi đáp án chỉ được khớp một finding nên dòng thứ hai bị tính là sót. Recall thật cao hơn con số trong bảng một chút.

Giới hạn của thực nghiệm

- Lỗi được cài nhân tạo vào diff nhỏ, dễ nhận ra hơn lỗi trong PR thật. Điều kiện này có lợi cho LLM.
- 53 case đủ để thấy xu hướng, chưa đủ để kết luận có ý nghĩa thống kê. Mỗi cấu hình mới chạy một đến hai lần.
- Tất cả lượt chạy dùng một model duy nhất (`gemini-3.1-flash-lite` qua API kiểu OpenAI). Đổi model có thể đổi kết quả.
- Chưa rà tay các finding bị tính là báo nhầm để tách phần bịa thật khỏi phần nhận xét đúng nhưng nằm ngoài đáp án.

---

5. Giai đoạn hiện tại

Dự án đã qua giai đoạn xây dựng và thực nghiệm chính. Còn lại là sửa một điểm yếu kỹ thuật do thực nghiệm chỉ ra, chạy thử với GitHub thật, và viết luận văn với số liệu mới nhất.

ĐÃ XONG	**Pipeline review và chế độ offline** Đã chạy đầu-cuối qua `app/cli.py` hàng trăm lượt trong thực nghiệm.
ĐÃ XONG	**Sandbox với năm công cụ static** Có integration test chạy container thật, bắt buộc mỗi công cụ phải báo ít nhất một phát hiện trên file có lỗi cài sẵn.
ĐÃ XONG	**LLM ba vai và self-verify** Chạy với Google Gemini qua backend kiểu OpenAI.
ĐÃ XONG	**Thực nghiệm bộ Python và bộ đa ngôn ngữ** 35 và 18 case, mỗi bộ ba cấu hình. Lượt `static` và `both` đã được chạy lại sau khi sửa ruff.
DỞ DANG	**Thực nghiệm tắt self-verify và lần chạy lặp** Đã chạy xong ngày 26/09 nhưng chưa commit (`eval/run_eval.py` đang sửa dở, `eval/out-noise/` chưa được theo dõi). `eval/score.py` chưa chấm cấu hình `llm_noverify`.
DỞ DANG	**Báo cáo tiến độ và luận văn** `BAO_CAO_TIEN_DO.md` và `eval/out/results.md` còn số liệu ngày 21/09: chưa có kết quả sau khi sửa ruff, chưa có bộ đa ngôn ngữ, chưa có phần self-verify. Nhận định "static là tập con của LLM" trong báo cáo không còn đúng.
DỞ DANG	**Kiểm thử tự động** Chạy `pytest` hôm nay: 4 unit test qua, 8 integration test bị bỏ qua vì Docker không chạy lúc kiểm tra. Chưa có test nào cho nhánh webhook và GitHub.

CHƯA LÀM

Chạy với GitHub App thật

File `.env` vẫn dùng mã App giả. `main.py`, `worker.py`, `github_app.py` và `memory.py` chưa từng chạy đầu-cuối với GitHub, Redis và Postgres thật. Cần đăng ký GitHub App và một đường hầm công khai.

CHƯA LÀM

Sửa bước gộp static với LLM

Nguyên nhân chính khiến cấu hình mặc định của sản phẩm kém hơn cấu hình chỉ LLM.

CHƯA LÀM

Backend Hermes Agent

Có mã (`HermesCLIBackend`) nhưng mọi lượt chạy đã ghi lại đều dùng backend kiểu OpenAI.

CHƯA LÀM

Bot học từ phản hồi

Ghi nhớ theo repo đã được đọc vào prompt, nhưng chưa có chỗ nào trong mã ghi vào nó. Comment của bot mời bấm 👍 / 👎 nhưng chưa có gì xử lý phản hồi đó.

Mốc thời gian

16-17/09	Dựng mã nguồn và môi trường, build image sandbox. Phát hiện semgrep chưa từng chạy được vì cần tải luật qua mạng; sửa bằng cách nạp sẵn luật vào image. Nối bandit vào script.
18/09	Phát hiện self-verify chưa từng chạy được do lỗi ghép chuỗi prompt; sửa và thêm test hồi quy. Tối ưu quét static từ khoảng 350 giây xuống 70–120 giây. Dựng bộ thực nghiệm.
21/09	Chạy xong 105 lượt trên bộ Python. Viết báo cáo tiến độ.
22-23/09	Thêm bộ đa ngôn ngữ. Commit đầu tiên.
25-26/09	Phát hiện ruff chưa từng trả về kết quả nào vì ghi cache vào thư mục chỉ đọc. Sửa, thêm cảnh báo khi công cụ thất bại và integration test, chạy lại `static` và `both`. Commit thứ hai.
26/09 (tối)	Chạy thực nghiệm tắt self-verify, lần chạy lặp của `llm`, và chạy lại bộ đa ngôn ngữ. Chưa commit.

Bài học lặp lại bốn lần: công cụ hỏng mà không ai biết

Semgrep, bandit, self-verify và ruff đều từng ở trạng thái không hoạt động mà không báo lỗi, vì lỗi bị nuốt và một lần chạy hỏng trông giống hệt một lần chạy không tìm thấy gì. Cả bốn lần đều lọt qua unit test. Dự án đã thêm ba hàng rào: script trong sandbox ghi tên công cụ thất bại ra file riêng, đầu mỗi báo cáo in trạng thái từng công cụ, và integration test chạy container thật. Hàng rào thứ nhất hiện áp dụng cho semgrep, gitleaks và ruff. Bandit và hadolint vẫn nuốt lỗi trong script và chỉ được integration test che chắn.

Hai chỗ tài liệu và mã chưa khớp

- Báo cáo và chú thích trong mã ghi webhook trả `202`. Mã hiện trả `200`, là mặc định của FastAPI. GitHub chấp nhận cả hai, nhưng nên sửa một trong hai cho thống nhất trước khi bảo vệ.
- Container sandbox vẫn chạy bằng user root và hệ thống file riêng của nó ghi được. Những gì được bảo vệ là mã nguồn (chỉ đọc), mạng (không có) và capability (bỏ hết). Trong mã có ghi chú về việc hạ xuống user thường.

Việc nên làm tiếp, theo thứ tự

1. **Sửa bước gộp static với LLM**, rồi chạy lại cấu hình `both`. Đây là thay đổi duy nhất có thể đảo lại kết luận về giả thuyết chính.
2. **Đưa số liệu mới vào tài liệu**: cho `score.py` chấm cả `llm_noverify`, sinh lại `results.md`, cập nhật báo cáo, commit các thay đổi đang dở.
3. **Quyết định về self-verify**: chạy lặp mỗi cấu hình vài lần để có khoảng dao động, hoặc thử dùng một model khác cho bước kiểm chứng. Kết quả nào cũng là nội dung đáng viết vào luận văn.
4. **Chạy thử với GitHub App thật** trên một repo thử nghiệm, để có ảnh chụp và bằng chứng cho phần demo.
5. **Rà tay các finding bị tính là báo nhầm** để báo cáo được precision gần với thực tế hơn.

---

6. Thuật ngữ

Pull request (PR)	Đề nghị gộp một nhóm thay đổi code vào nhánh chính, là nơi diễn ra việc review.
Diff	Phần khác nhau giữa bản cũ và bản mới của các file trong PR.
Webhook	Cách GitHub chủ động gọi vào hệ thống khi có sự kiện.
HMAC	Mã xác thực tính từ nội dung và một khoá bí mật, chứng minh webhook thật sự đến từ GitHub.
Check run	Dòng trạng thái hiện trên PR (đang chạy, đạt, không đạt). Có thể dùng để chặn merge.
Static analysis	Phân tích mã nguồn mà không chạy chương trình.
Sandbox	Môi trường cô lập để xử lý thứ không đáng tin.
Finding	Một phát hiện của bot: file, dòng, mức độ, loại lỗi, giải thích, gợi ý sửa, độ tin cậy.
Hallucination	LLM tạo ra thông tin nghe hợp lý nhưng sai. Ở đây là bịa ra lỗi không tồn tại hoặc báo sai dòng.
Self-verify	Lượt gọi LLM thứ hai để kiểm chứng lại kết quả của lượt đầu.
Ground truth	Đáp án đã biết trước, dùng để chấm điểm.
Precision	Trong những gì bot báo, bao nhiêu phần là đúng.
Recall	Trong những lỗi có thật, bot bắt được bao nhiêu phần.
F1	Trung bình điều hoà của precision và recall. Chỉ cao khi cả hai cùng cao.

Tổng hợp từ mã nguồn, lịch sử git và các file kết quả thực nghiệm của repo HermesQA tại thời điểm 01/10/2026.